

Кравченко А.С.
ДОКУМЕНТАЦИЯ К ПРОЕКТУ
“ОБЛАЧНОЕ ХРАНИЛИЩЕ ИЗОБРАЖЕНИЙ”
РАЗРАБОТАННЫЙ НА ОСНОВЕ DRF И PyQt5

ЮЖНО-САХАЛИНСК 2026

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	3
2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	4
3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ	6
4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ	7
5. ПЕРЕЧЕНЬ ВХОДНЫХ ПЕРЕМЕННЫХ	7
6. ПЕРЕЧЕНЬ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ	8
7. КАК РАБОТАЕТ ПРОГРАММА	8
8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ	10
9. ЛИСТИНГ ПРОГРАММЫ	11

1. ВВЕДЕНИЕ

Документация описывает разработанное программное обеспечение для передачи и получения изображений по сети с использованием клиент-серверной архитектуры. Проект реализован на языке Python с применением фреймворков PyQt5 и Django REST Framework.

Программное обеспечение состоит из двух основных частей:

1. Серверная часть на базе Django REST Framework:

- обработка HTTP-запросов;
- хранение изображений;
- предоставление REST API;
- сериализация данных.

2. Клиентская часть на базе PyQt5:

- графический интерфейс пользователя;
- выбор изображения;
- отправка изображений на сервер;
- получение списка изображений;
- отображение изображений.

Программа реализует взаимодействие между клиентом и сервером через HTTP-запросы библиотеки requests.

2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1. ВВЕДЕНИЕ

Наименование разработки: «Система передачи изображений».

Программа предназначена для демонстрации клиент-серверного взаимодействия между приложением PyQt5 и сервером Django REST Framework.

2.2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

Программа предназначена для:

- загрузки изображений на сервер;
- хранения изображений;
- получения списка загруженных изображений;
- просмотра изображений через графический интерфейс.

2.3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

2.3.1. Требования к функциональным характеристикам

2.3.1.1. Требования к инструментальной среде

1. Python версии не ниже 3.10.
2. Используемые библиотеки:
3. PyQt5;
4. Django;
5. Django REST Framework;
6. Pillow;
7. requests.

2.3.1.2. Требования к пользовательскому интерфейсу

1. Интерфейс должен содержать:
 - а. кнопки отправки и получения изображений;
 - б. поле ввода пароля;

- c. список изображений;
 - d. область предпросмотра изображения.
- 2. Пользователь должен иметь возможность:
 - a. выбрать изображение;
 - b. отправить изображение;
 - c. получить изображения с сервера;
 - d. просмотреть выбранное изображение.

2.3.1.3. Требования к реализуемым функциям

- 1. При запуске клиентское приложение:
 - a. создает графический интерфейс;
 - b. подключает обработчики событий.
- 2. При выборе изображения:
 - a. открывается QFileDialog;
 - b. путь сохраняется в переменную `image_path`.
- 3. При отправке изображения:
 - a. выполняется POST-запрос к серверу;
 - b. изображение передается через `multipart/form-data`.
- 4. При получении изображений:
 - a. выполняется GET-запрос;
 - b. список изображений отображается в `QListWidget`.
- 5. При выборе изображения из списка:
 - a. изображение загружается;
 - b. выполняется отображение в `QLabel`.

2.3.2. Требования к надежности

- 1. Программа должна обрабатывать ошибки:
 - a. отсутствия изображения;
 - b. отсутствия соединения с сервером;
 - c. пустого списка изображений.
- 2. Интерфейс не должен завершаться аварийно при ошибках запросов.

2.3.3. Требования к составу технических средств

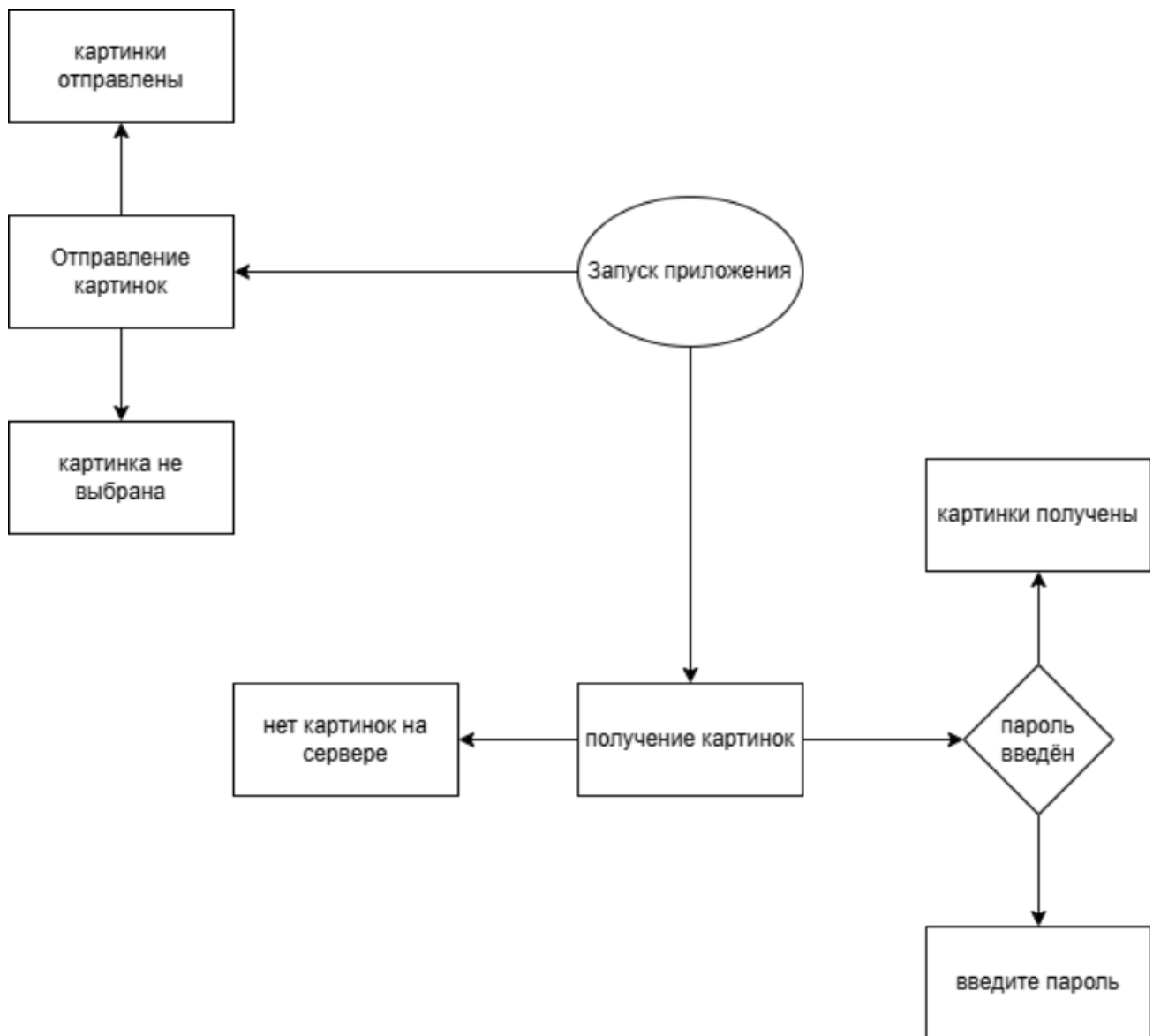
- 1. ОС Windows/Linux.
- 2. Наличие Python и необходимых библиотек.
- 3. Наличие сетевого соединения между клиентом и сервером.

2.3.4. Требования к совместимости

- 1. Сервер должен поддерживать HTTP-запросы.

2. Передача данных выполняется через REST API.

3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

MyWindow(QWidget)	главное окно программы
initUI()	Создание интерфейса
change_text1()	Оправка изображения
change_text2()	Получение изображений

<code>item_clicked()</code>	Выбор изображения
<code>pick_file()</code>	Выбор файла
<code>Image(models.Model)</code>	Мдель изображения
<code>ImageSerializer</code>	Сериализация данных
<code>requests.post()</code>	Оправка POST-Запроса
<code>requests.get()</code>	Отправка GET-Запроса

5. ПЕРЕЧЕНЬ ВХОДНЫХ ПЕРЕМЕННЫХ

<code>self.password_field.text()</code>	Введённый пароль
<code>self.image_path</code>	Путь к изображению
<code>item.text()</code>	URL изображения
<code>r.content</code>	Содержимое ответа сервера
<code>files={"image": f}</code>	Отправляемое изображение

6. ПЕРЕЧЕНЬ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

<code>self.widget3.setText()</code>	Вывод статуса
<code>self.image.setPixmap()</code>	Отображение изображения
<code>QListWidgetItem()</code>	Добавление изображения в список
<code>requests.post()</code>	Отправка изображения
<code>requests.get()</code>	Получение изображений

7. КАК РАБОТАЕТ ПРОГРАММА

7.1. Запуск программы

При запуске выполняются:

1. Запуск сервера Django:
`os.system('python project/run.py')`
2. Создание QApplication.

3. Создание окна MyWindow.
4. Отображение интерфейса.

7.2. Выбор изображения

Пользователь нажимает кнопку:

```
self.image_button.clicked.connect(self.pick_file)
```

После чего:

- открывается QFileDialog;
- выбирается изображение;
- путь сохраняется в self.image_path.

7.3. Отправка изображения

Метод:

```
change_text1()
```

выполняет:

1. Проверку пароля.
2. Открытие файла изображения.
3. POST-запрос:

```
requests.post(  
    "http://127.0.0.1:8000/main/images/",  
    files={"image": f}  
)
```

4. Сервер сохраняет изображение.

7.4. Получение изображений

Метод:

```
change_text2()
```

выполняет:

1. GET-запрос:
`requests.get("http://127.0.0.1:8000/main/images/")`

2. Получение JSON-списка изображений.
3. Загрузка изображений.
4. Добавление элементов в `QListWidget`.

7.5. Просмотр изображения

При выборе изображения:

```
self.images_group.itemClicked.connect(self.item_clicked)
```

выполняется:

- загрузка изображения по URL;
- отображение в `QLabel`.

8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ

8.1. Архитектура программы

Программа построена по клиент-серверной архитектуре:

Клиент:

- PyQt5 GUI;
- `requests`;
- обработка изображений.

Сервер:

- Django;
- Django REST Framework;
- `ImageField`;
- сериализация JSON.

8.2. Взаимодействие компонентов

Отправка изображения

PyQt5 → `requests.post()` → Django REST API → Image модель

Получение изображения

PyQt5 → `requests.get()` → Django REST API → JSON → `QListWidget`

8.3. Работа интерфейса

GUI работает через механизм сигналов и слотов PyQt5:

```
clicked.connect(...)
itemClicked.connect(...)
```

9. ЛИСТИНГ ПРОГРАММЫ

9.1. Модель Django

```
class Image(models.Model):
    image = models.ImageField(upload_to='images/')
```

9.2. Сериализатор

```
class ImageSerializer(serializers.ModelSerializer):
    class Meta:
        model = Image
        fields = ['image']
```

9.3. Отправка изображения

```
r = requests.post(
    "http://127.0.0.1:8000/main/images/",
    files={"image": f}
)
```

9.4. Получение изображений

```
r = requests.get(
    "http://127.0.0.1:8000/main/images/"
)
```

9.5. Отображение изображения

```
self.pixmap.loadFromData(x.content)
scaled_pixmap = self.pixmap.scaled(
    200,
    200,
    QtCore.Qt.KeepAspectRatio
)
self.image.setPixmap(scaled_pixmap)
```